

GBF缓存功能

背景

需求分析

显性需求

- 业务中出现一个流程（Session）对同一下游Fetcher的多次请求调用
 - 参数一样，数据也一样
 - 在流程中无法收敛为一次执行
 - 期望避免流量放大
 - 能够加速整个流程的执行
- 一些常用不变的数据期望长时间缓存在内存中
 - 减少下游访问
 - 执行过程提速
 - 地区列表，邮编等数据

隐性需求

- 简单易用的缓存功能
 - 定义简单
 - 使用方便
- 可根据不同场景定制缓存参数
 - 异步写
 - 启动初始化
 - 读写拷贝
- 能够用不同的缓存进行组合使用
 - Session本地缓存
 - 本地内存缓存
 - redis等中间件缓存
 - 按优先级获取组合使用
 - 框架提供单缓存Provider，业务方可根据实际场景自定义组合缓存的Provider
- 能够对缓存进行管控
 - 统计缓存指标和日志
 - 监控缓存运行状况

- 预警功能
- 不能或尽量减少对业务过程的影响
 - Session缓存需默认淘汰时间
 - 整个Session缓存释放可异步
 - 缓存过程的异常在在内部catch打印，不抛到业务中
 - 避免将缓存数据存放在业务过程数据中
 - Pipeline上下文
 - Session会话
 - Request、Result出入参

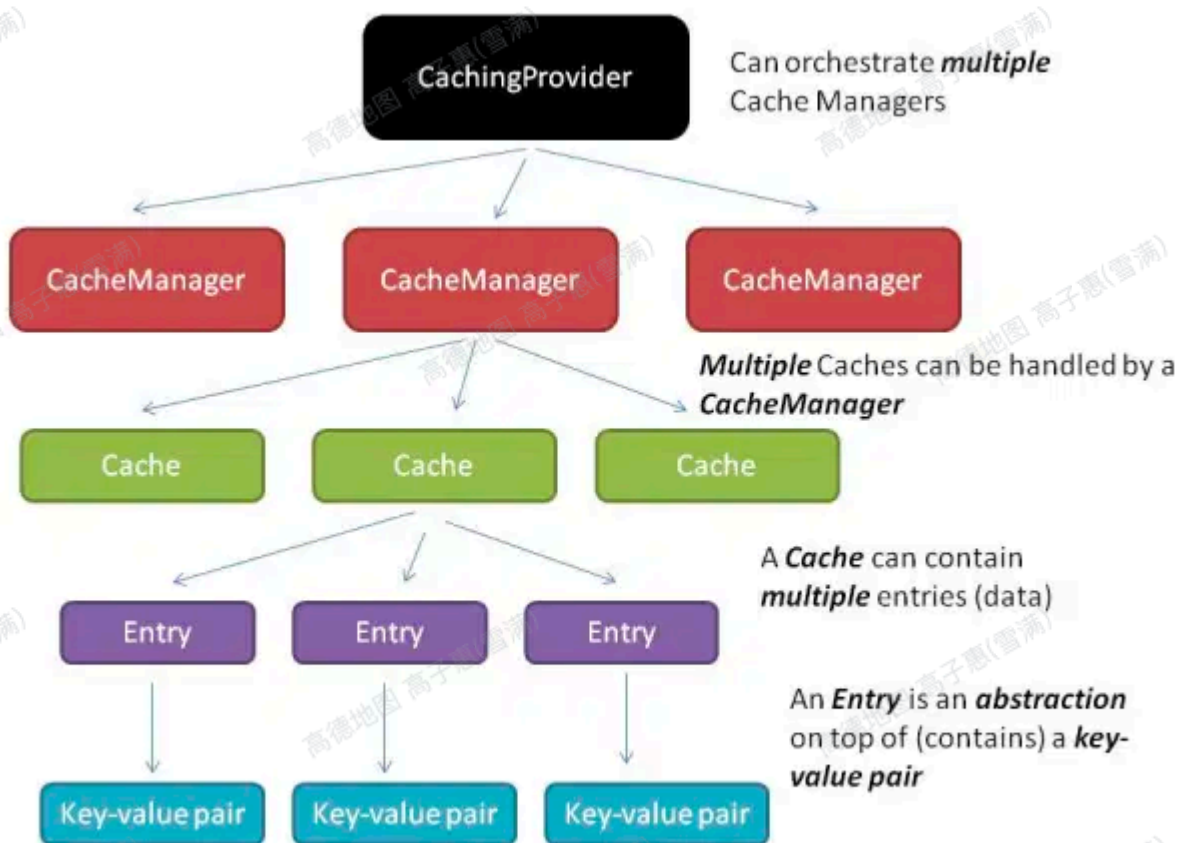
设计目标

- 标准易用的缓存体系
 - JSR 107标准
 - 沿用规则扩展的使用习惯（字段上注解）
- 多种实现可用
 - Session
 - Caffeine
 - ...
- 支持业务自定义组合使用场景
 - 定制优先级
 - 多种配置同一实例
- 性能可靠
 - 高并发
 - 内存占用
 - 指标监控

设计

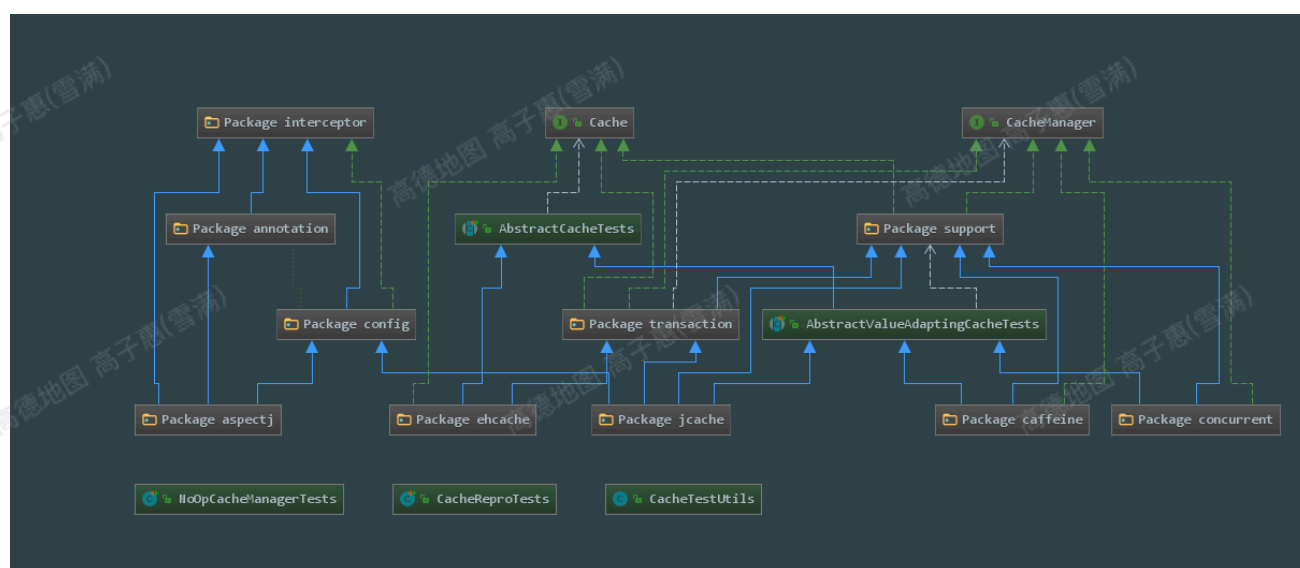
技术调研

JSR 107标准



一套标准的java缓存结构和接口标准，大多缓存库均有基于该标准的适配方案；但同时因为严格按照其实现会存在使用和执行效率的牺牲，如Caffeine就参考但未完全照搬其标准。

SpringCache



Spring 3.1之后，提供了基于注解的缓存支持，和spring提供的其它服务类似，spring cache对缓存进行了抽象，提供了统一的cache操作的语义（如加入cache、删除cache、更新cache等），使用者可

以在spring cache的基础上，进行扩展，使用自己的cache系统作为底层cache存储实现。

同时spring也提供了一些默认的实现，如基于ConcurrentMap实现的缓存，Ehcache 2.x，Caffeine Cache（类似于Guava的Cache，在spring 4.X中为GuavaCache）等。

另外Spring不进行Cache的缓存策略的维护，如缓存过期时间等，这些都是由底层Cache自己实现，Spring只是提供了一个Wrapper，提供一套对外一致的API。

优缺点

• 优点

- 支持开箱即用（Out Of The Box），并提供基本的Cache抽象，方便切换各种底层Cache
- 兼容JSR 107标准，理论上基于该标准的Cache实现都可以继承到SpringCache体系
- 通过Cache注解即可实现缓存逻辑透明化，让开发者关注业务逻辑
- 支持比较复杂的缓存逻辑（Spel表达式和自定义Manger等）

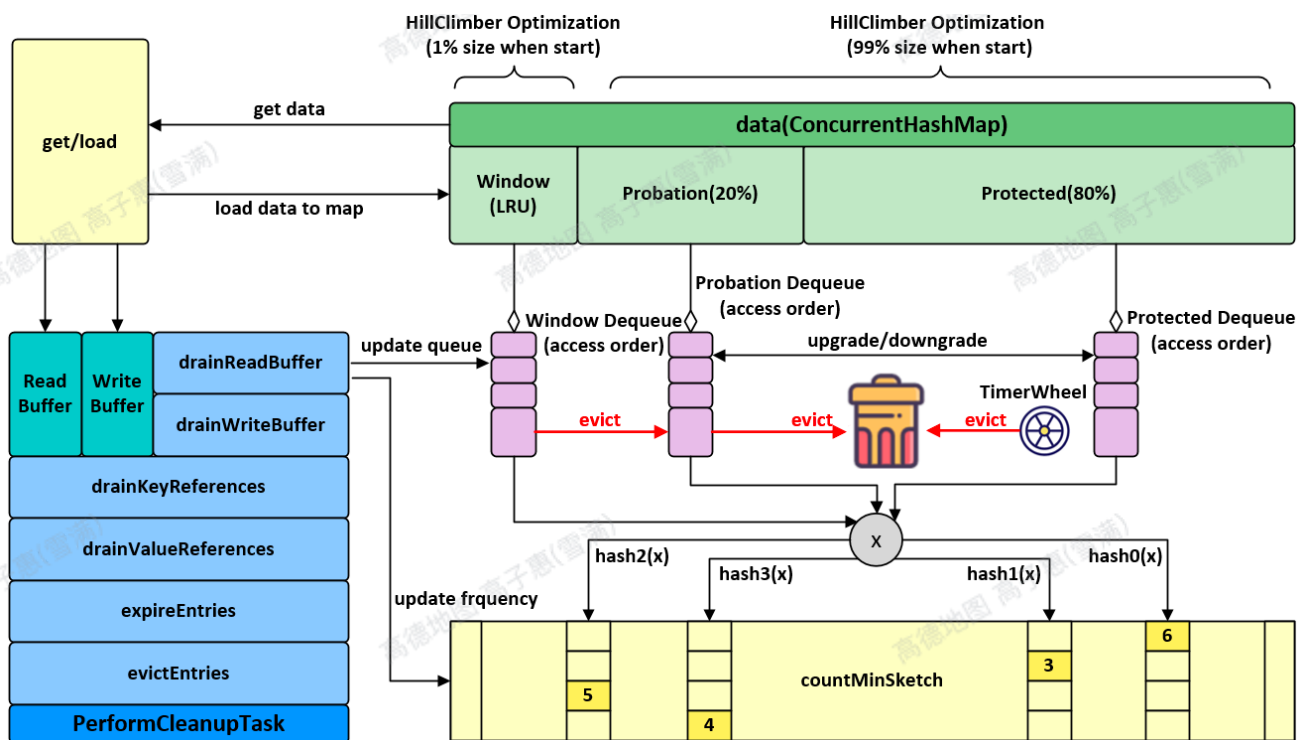
• 缺点

- 针对多场景的缓存配置比较麻烦，需要自定义Manager
- Spel表达式在高并发场景会成为瓶颈
- 另外Spring Cache抽象的操作中没有锁的概念，当多线程并发操作（更新或者删除）同一个缓存项时，有可能读取到过期的数据
- 内部调用，在非 public 方法上使用注解，会导致缓存无效。由于 SpringCache 是基于 Spring AOP 的动态代理实现，由于代理本身的问题，当在同一个类中调用另一个方法，会导致另一个方法的缓存不能使用，这个在编码上需要注意，避免在同一个类中这样调用
- 不支持多key操作，由于是基于方法注解实现，return结果只有一个，因此不支持批量操作

Caffeine

Caffeine是一个基于Java8开发的提供了[近乎最佳](#)命中率的[高性能](#)的缓存库。

缓存和[ConcurrentMap](#)有点相似，但还是有所区别。最根本的区别是[ConcurrentMap](#)将会持有所有加入到缓存当中的元素，直到它们被从缓存当中手动移除。但是，Caffeine的缓存 `Caffeine` 通常会被配置成自动驱逐缓存中的元素，以限制其内存占用。在某些场景下，`LoadingCache` 和 `AsyncLoadingCache` 因为其自动加载缓存的能力将会变得非常实用。

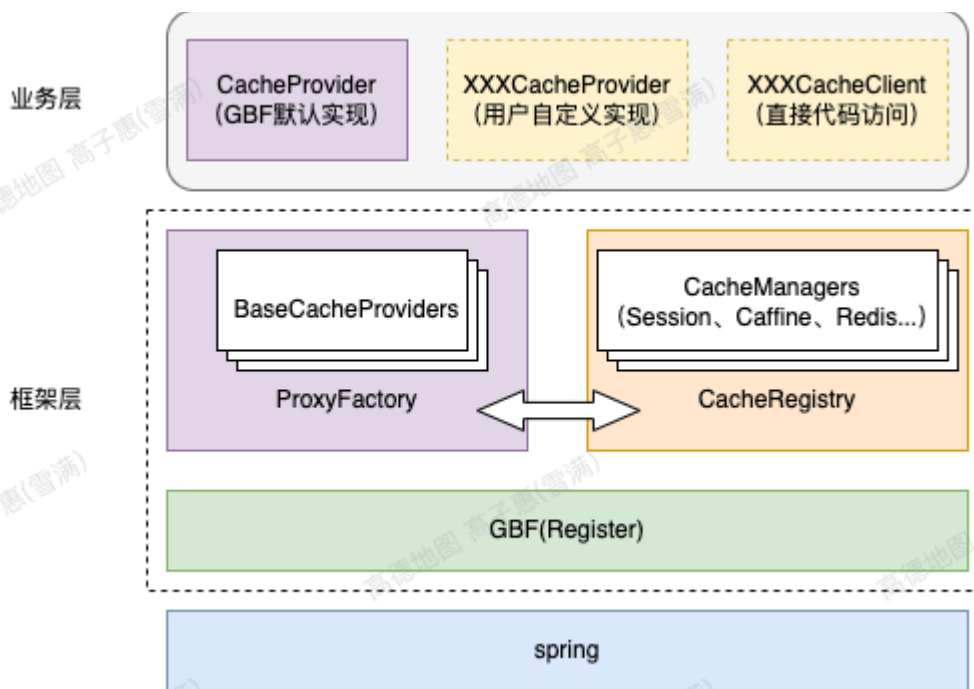


优缺点

- 优点
 - 效率最佳
 - 灵活易上手
 - 适用度广
- 缺点
 - 需要自己创建维护bean
 - 不完全符合JSR107规范

概要设计

总体设计

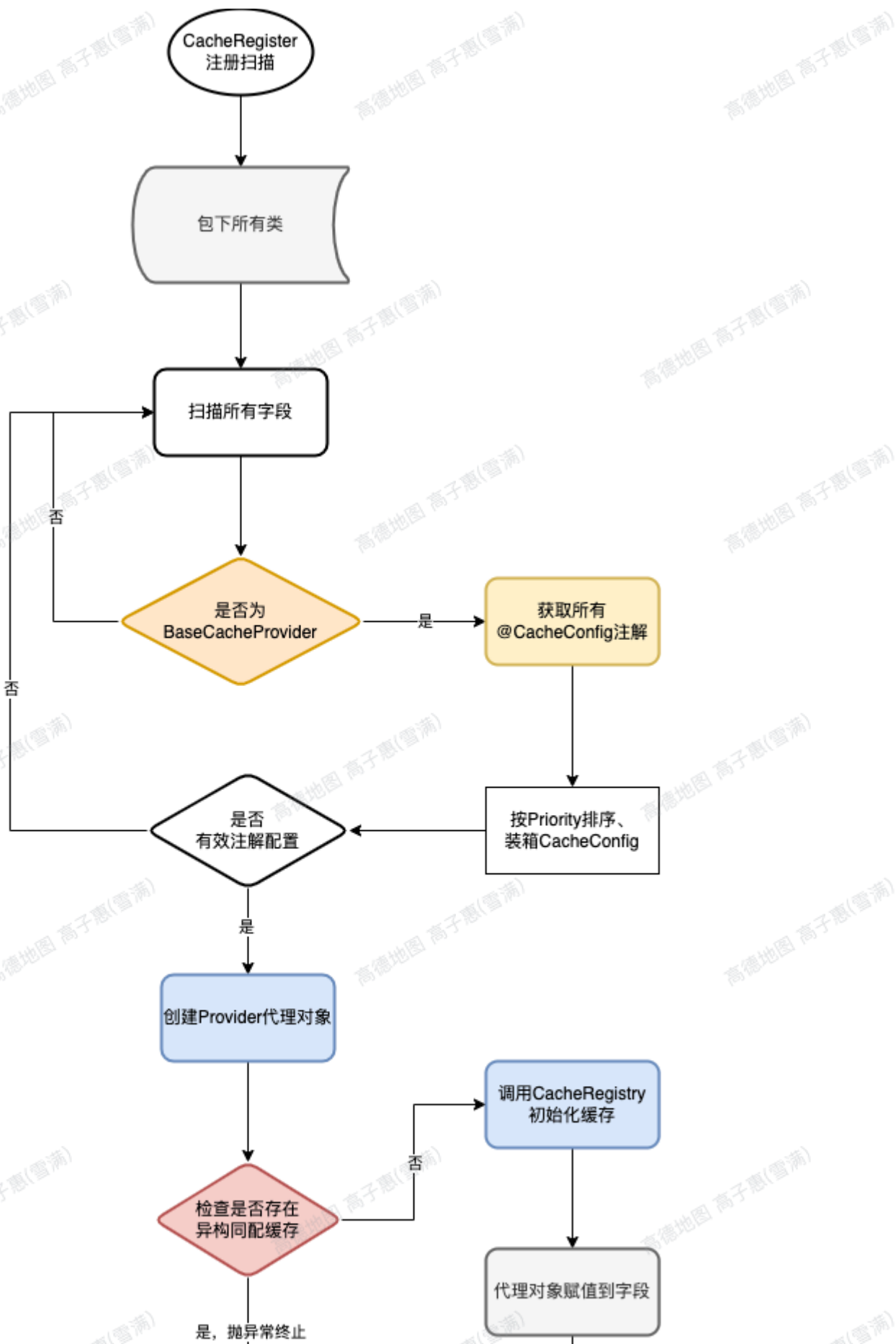


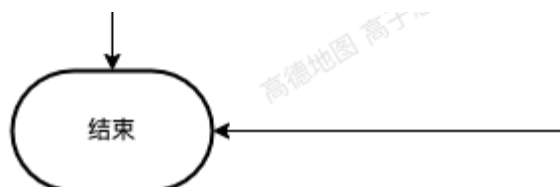
基于上面的调研和框架自身的使用情况，缓存功能按如上进行总体设计，主要有下面几点：

- 缓存功能的使用基于GBF现有的生命周期进行实现
 - Register启动扫描包初始化注解的字段对应的Provider实例
 - 创建对应的Registry存储所有缓存Manager以及其下的各种缓存实例
 - 通过实例化后的Provider对象赋值到声明字段来让业务声明后直接用
- 缓存的分层参照JSR107规范进行设计
 - Provider：业务定义和调用的关联点（可扩展实现多缓存组合使用功能）
 - Manager：缓存实例管理器（包含指标统计功能、Session回调销毁等）
 - Cache：缓存实例和缓存管理器（实现了对三方缓存功能的封装）
- 面向接口和扩展设计，业务用户在每一层均可实现基于自有逻辑的特定实现

关键设计

启动流程

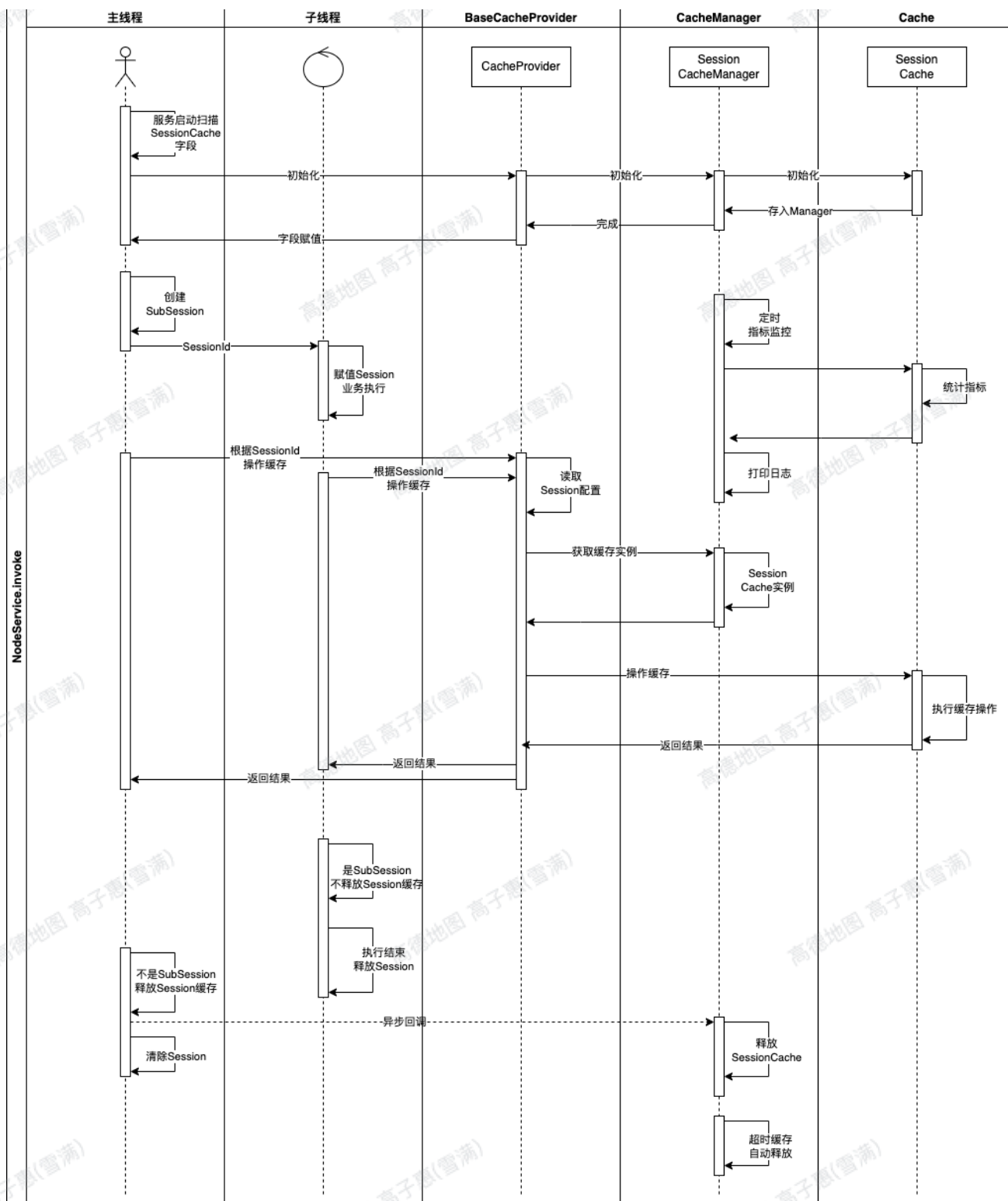




基于GBF现有的启动扫描和注册机制：

- 包含合法性检查
 - 注解检查
 - 同配异构体检查
- 可根据配置选择InitOnStartup
- 可自定义基于BaseProvider实现多缓存配置，并被GBF扫描初始化

Session缓存



Session缓存的时序周期：

- 启动时随系统扫类根据注解配置创建实例对象
- 根据Session和子Session执行过程所携带的id来对Session缓存进行操作
- Session结束后缓存销毁有三种策略
 - Session销毁时的异步回调（业务无感知）

- ## 详细设计



```

public V computeIfAbsent(K key, Function<? super K, ? extends V>
mappingFunction) {
    if (key == null || mappingFunction == null)
        throw new NullPointerException();
    int h = spread(key.hashCode());
    V val = null;
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & h)) == null) {
            Node<K,V> r = new ReservationNode<K,V>();
            synchronized (r) {
                if (casTabAt(tab, i, null, r)) {
                    binCount = 1;
                    Node<K,V> node = null;
                    try {
                        if ((val = mappingFunction.apply(key)) != null)
                            node = new Node<K,V>(h, key, val, null);
                    } finally {
                        setTabAt(tab, i, node);
                    }
                }
            }
            if (binCount != 0)
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            boolean added = false;
            synchronized (f) {
                if (tabAt(tab, i) == f) {
                    if (fh >= 0) {
                        binCount = 1;
                        for (Node<K,V> e = f;; ++binCount) {
                            K ek; V ev;
                            if (e.hash == h &&
                                ((ek = e.key) == key ||
                                 (ek != null && key.equals(ek)))) {
                                val = e.val;
                                break;
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        Node<K,V> pred = e;
        if ((e = e.next) == null) {
            if ((val = mappingFunction.apply(key)) !=
null) {
                added = true;
                pred.next = new Node<K,V>(h, key, val,
null);
            }
            break;
        }
    }
}
else if (f instanceof TreeBin) {
    binCount = 2;
    TreeBin<K,V> t = (TreeBin<K,V>)f;
    TreeNode<K,V> r, p;
    if ((r = t.root) != null &&
        (p = r.findTreeNode(h, key, null)) != null)
        val = p.val;
    else if ((val = mappingFunction.apply(key)) !=
null) {
        added = true;
        t.putTreeVal(h, key, val);
    }
}
}
if (binCount != 0) {
    if (binCount >= TREEIFY_THRESHOLD)
        treeifyBin(tab, i);
    if (!added)
        return val;
    break;
}
}
if (val != null)
    addCount(1L, binCount);
return val;
}

```

其他

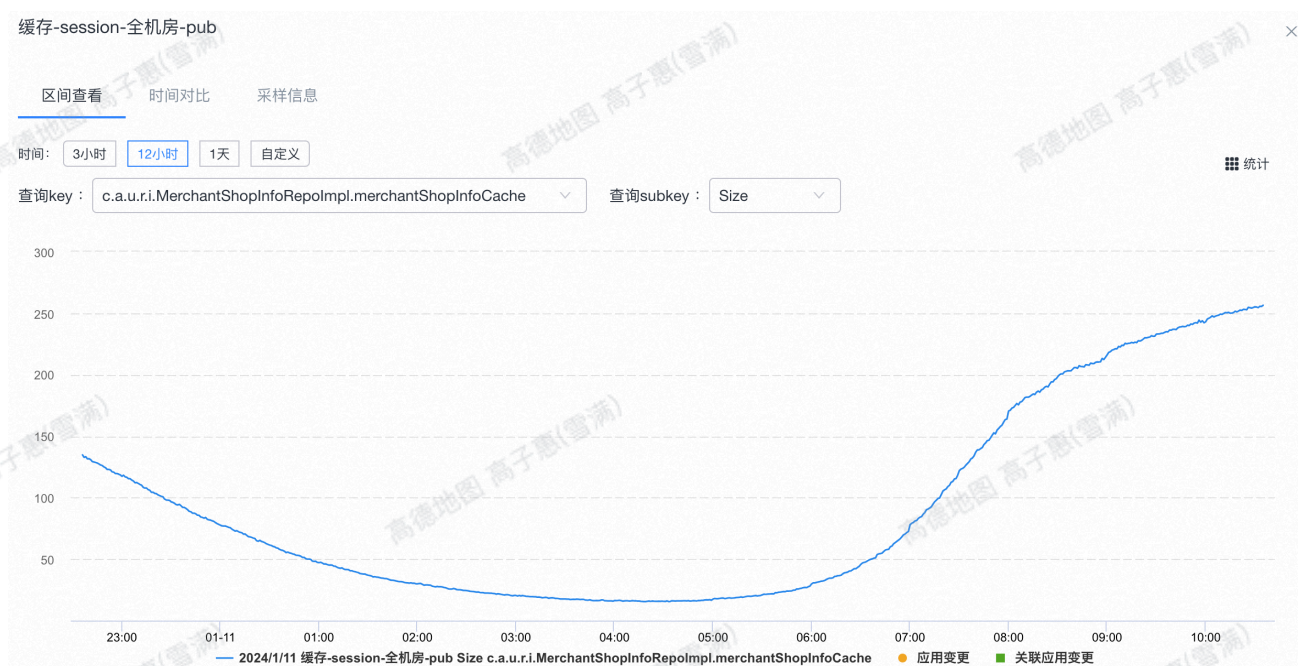
测试建议

- 对服务进行压测
 - 观察CPU、内存变化、GC变化：[监控视图 \(alibaba-inc.com\)](#)
 - 观察RT变化、错误变化等：[GBF-US业务平台_关键指标 \(alibaba-inc.com\)](#)
 - 观察监控指标：[缓存-session-全机房-gray \(alibaba-inc.com\)](#)
- 对服务接口进行diff
 - 重点关注：Action没找到执行报错、mif条件（adcodein/notin）报错
 - 重点接口：本次缓存升级涉及的接口
- 验证Fetcher日志中的process字段
- 探索测试：session缓存相关的其他可能问题

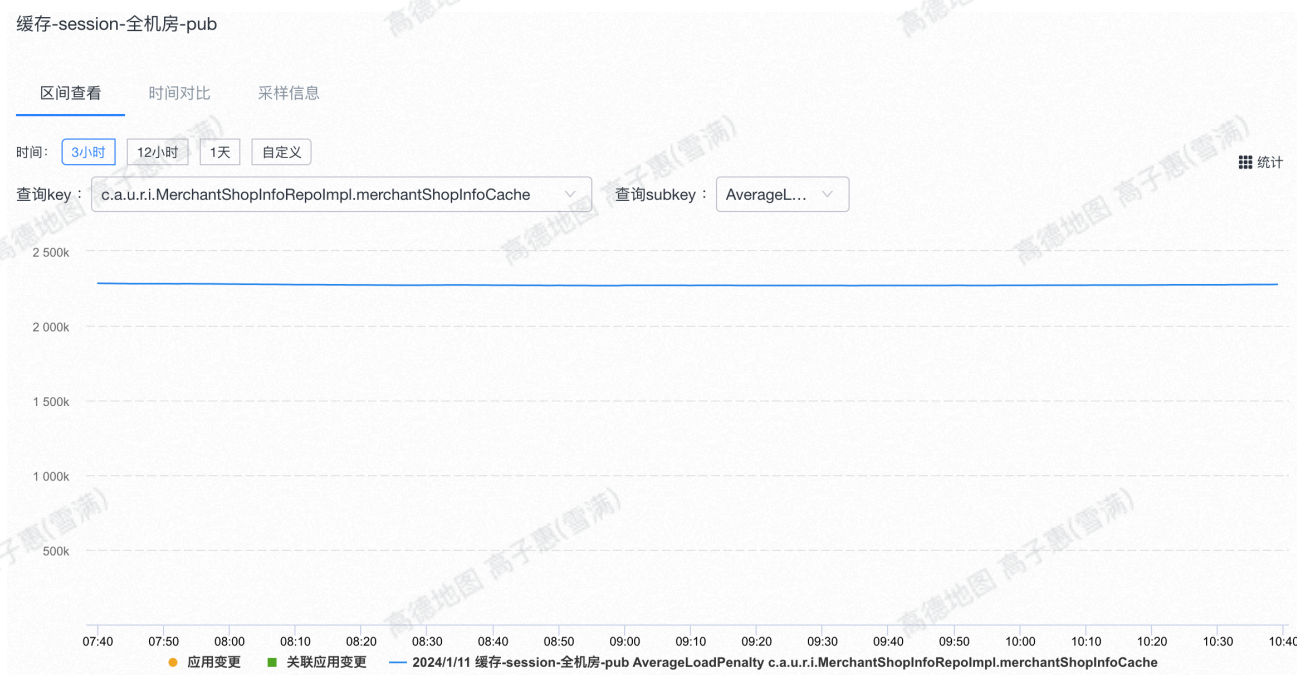
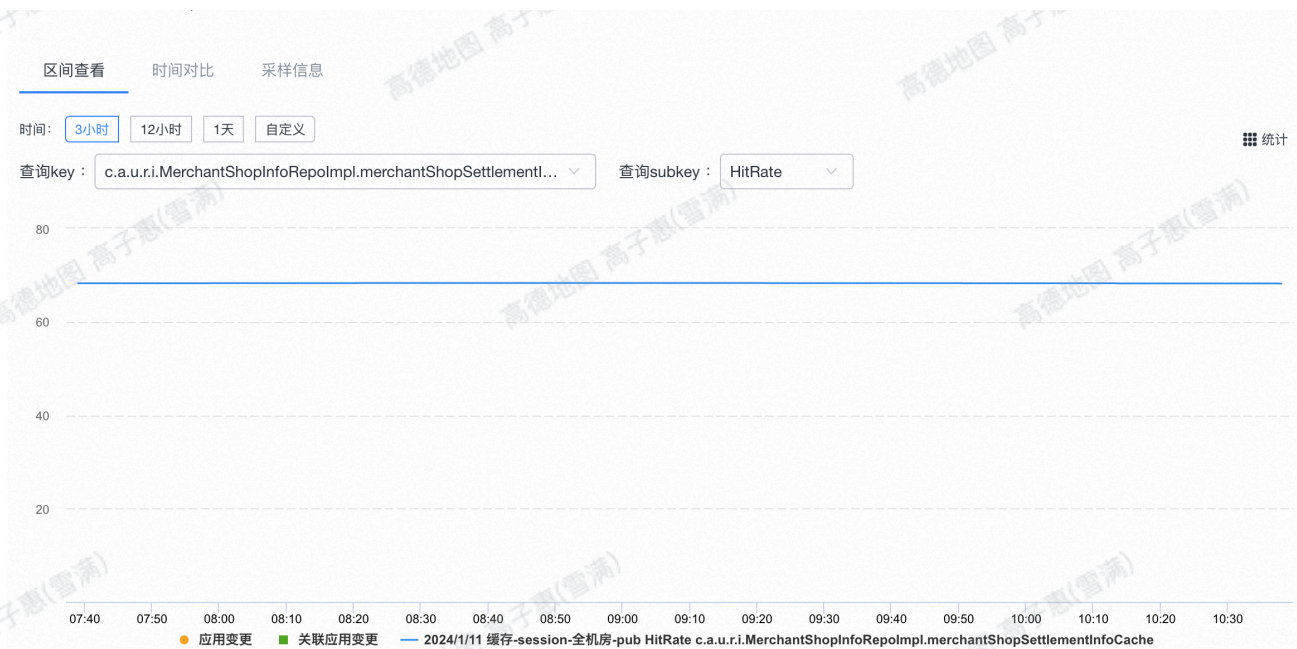
线上验证

缓存指标

从上线后缓存的容量变化看，其容量随着每天早晚高峰平滑涨跌，未触达最大容量上限，表现正常

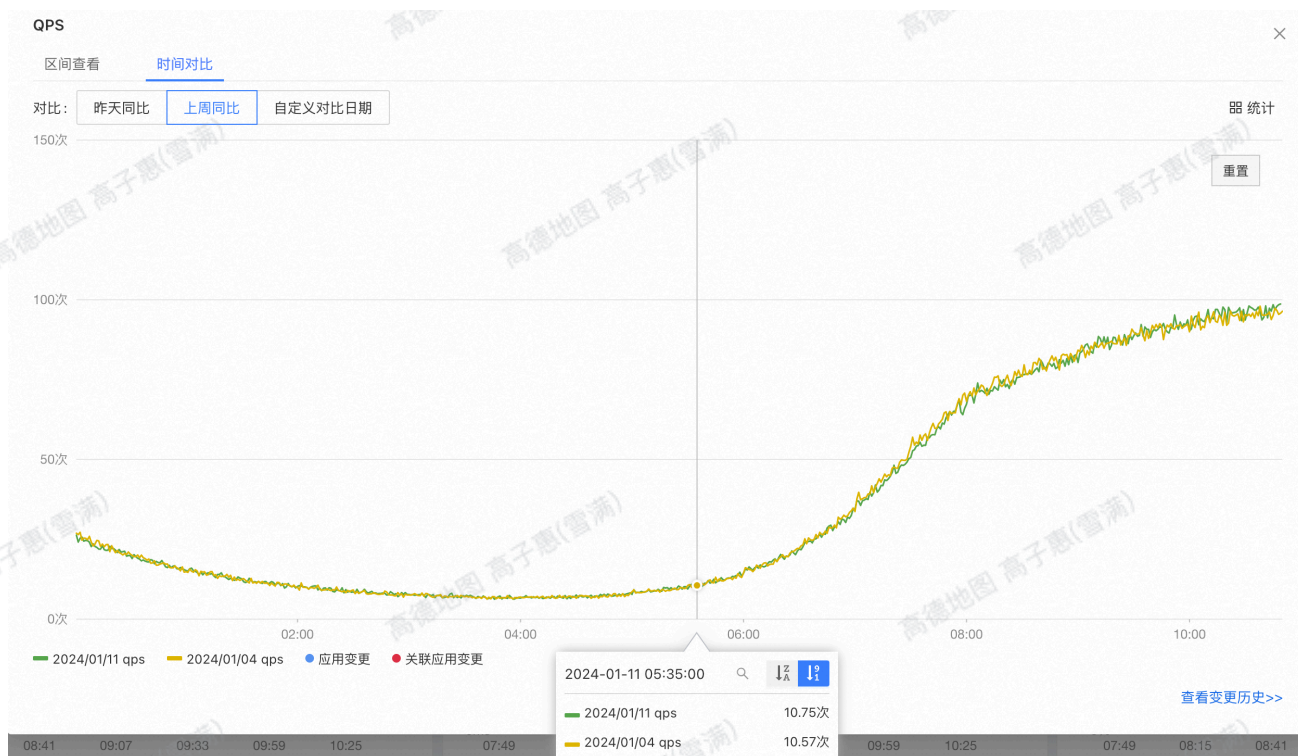


缓存命中率和加载率等计算指标也都维持在一个均值水平，无大幅波动

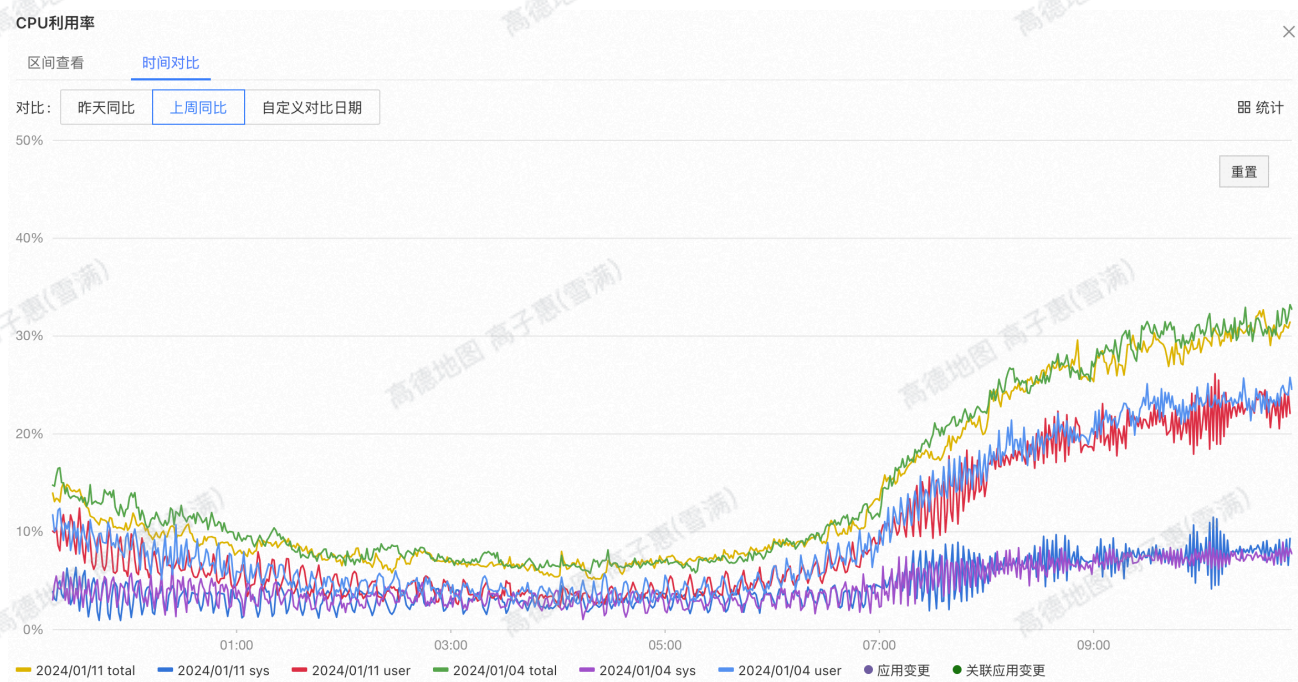


机器指标

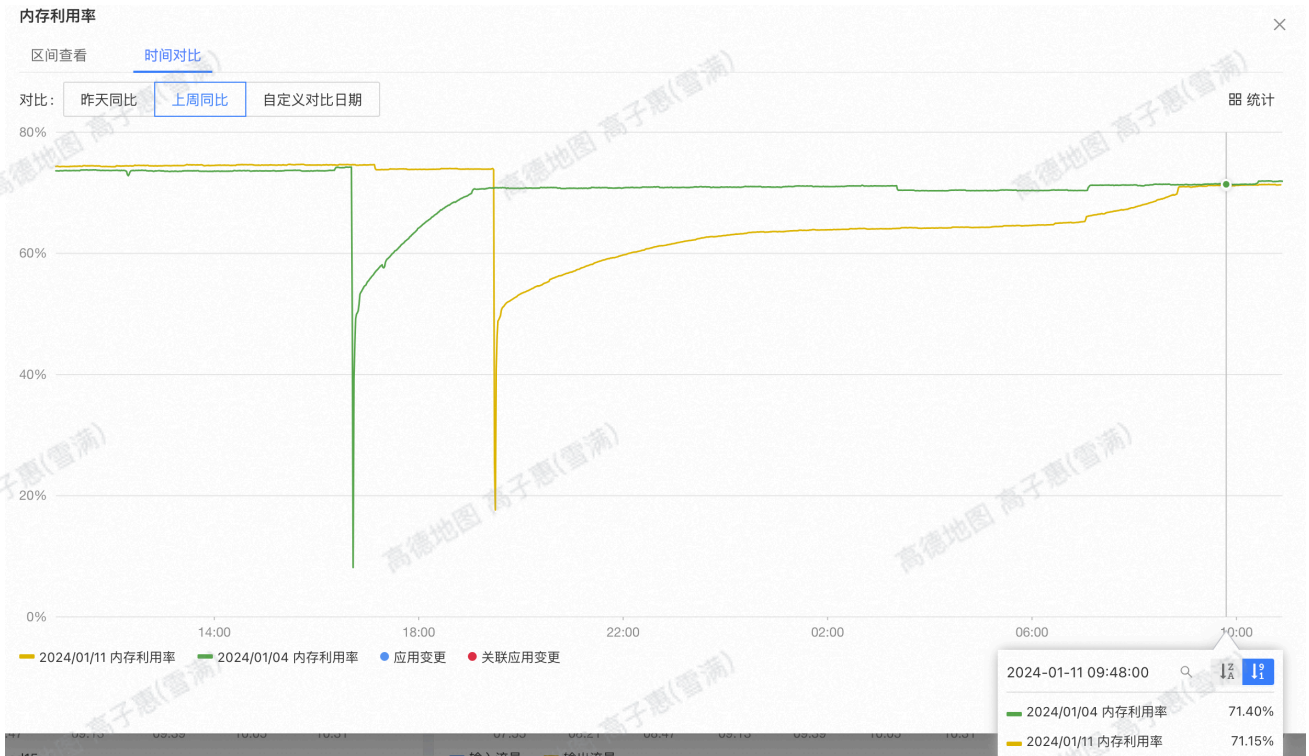
在上周同QPS的情况下



cpu较基线没有明显改变



内存较基线没有明显改变

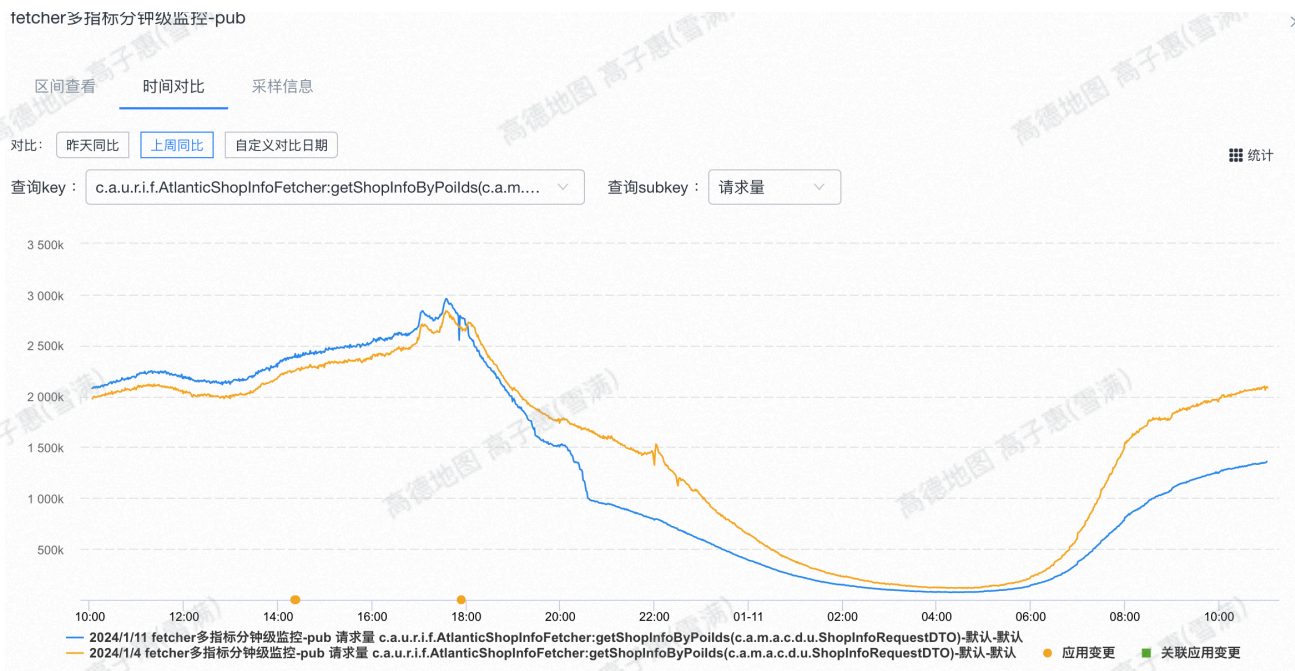


GC均值有所下降，分析这是由于优化了trace日志每次打印创建LogDetail对象带来的现象



Fetcher请求量

以c.a.u.r.i.MerchantShopInfoRepoImpl.merchantShopInfoCache对应的
c.a.u.r.i.f.AtlanticShopInfoFetcher:getShopInfoByPoilds(c.a.m.a.c.d.u.ShopInfoRequestDTO)为例



请求量有了明显的下降（由于该Fetcher以前用了线程内的临时session缓存方案，请求量降幅与缓存命中率不等）

流程RT

对应上面的Fetcher，shopInfo流程的RT也有了相应的降幅，最大降4ms，均值降了1ms



结论：由于异步节点以及同步加载机制的原因，session缓存带来的主要收益是下游Fetcher访问次数的减少，经过观察符合预期，没有带来其他额外资源负担。

参考文档

JCache: [jsr107/jsr107spec: JSR107 Cache Specification \(github.com\)](https://github.com/jsr107/jsr107spec)

SpringCache Doc: [IO \(spring.io\)](https://spring.io)

SpringCache github: [spring-guides/gs-caching: Caching Data with Spring :: Learn how to cache data in memory with Spring \(github.com\)](https://spring-guides.github.io/gs-caching/)

Caffine github: [ben-manes/caffeine: A high performance caching library for Java \(github.com\)](https://github.com/ben-manes/caffeine)

springCache原理详解: [Spring cache原理详解 - 掘金 \(juejin.cn\)](http://juejin.cn)

SpringCache简介: [spring cache 简介 - ATA \(atatech.org\)](http://ata.tech)